Web App Bug Fix Worklog

Pre-requisites:

- Windows 10 or later or Windows Server 2016 or later
- An Intel Pentium 4 processor or later that is SSE3 capable

Starting Point:

At the beginning of 8/10/2025 there were various bugs that needed immediate fixing in order to perform further work on the authentication functionality of the backend. These bugs can be summed up with the console output from the browser running the web app as seen below.

Backend URL: https://api.lab.crusaders.clockwork.vip

cognito-idp.ap-south...-2.amazonaws.com/:1

Failed to load resource: the server responded with a status of 400 ()

api.lab.crusaders.cl...ip/homepage/notes:1

Failed to load resource: the server responded with a status of 502 (Bad Gateway)

Home.tsx:96 Error fetching all notes:

ga

api.lab.crusaders.cl...e/documents/total:1

Failed to load resource: the server responded with a status of 502 (Bad Gateway)

Home.tsx:122 Error during data fetching:

ga

(anonymous) @ Home.tsx:122

api.lab.crusaders.cl...ents/most-popular:1

Failed to load resource: the server responded with a status of 502 (Bad Gateway)

api.lab.crusaders.cl.../recently-created:1

Failed to load resource: the server responded with a status of 502 (Bad Gateway)

cognito-identity.ap-...-2.amazonaws.com/:1

Failed to load resource: the server responded with a status of 400 ()

api.lab.crusaders.cl...ip/homepage/notes:1

Failed to load resource: the server responded with a status of 502 (Bad Gateway)

useFetchUser.ts:35 Error fetching user info: ResourceNotFoundException: IdentityPool 'ap-southeast-2:3d72b6f1-a857-4148-8f01-ae3ab2c24c9b' not found.

at async useFetchUser.ts:21:15

Problem 1: Unresponsive API (502 – Bad Gateway)

This issue was indicated by the number of issues returning "Failed to load resource: the server responded with a status of 502 (Bad Gateway)" in the console. This problem is characterized by the API gateway being unable to reach the backend service due to misconfiguration.

This issue was resolved by connecting to the backend server "ClockWorkWeb" via SSH and reviewing and fixing certain nginx configuration files. These files include:

- /etc/nginx/conf.d/clockwork-api.conf
- /etc/nginx/conf.d/api-lab-clockwork.conf

By reviewing these files, it was uncovered that they both contained configuration code relating to the "Lab" branch of the web application. As a result, this caused a conflict for the server, and consequently the API broke.

To resolve this, the "Lab" branch code from clockwork-api.conf was commented out as seen below:

```
# Production environment

server {

  listen 80;

  server_name api.prod.clockwork.vip;
```

```
    location /.well-known/acme-challenge/ {
        root /var/www/html;
    }

    location / {
        proxy_pass http://localhost:8800;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# Staging environment
server {
    listen 80;
    server_name api.staging.clockwork.vip;

    location /.well-known/acme-challenge/ {
        root /var/www/html;
    }

    location / {
        proxy_pass http://localhost:8801;
```

```
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# Lab environment
#server {
#    listen 80;
#    server_name api.lab.clockwork.vip;
#
#    location /.well-known/acme-challenge/ {
#        root /var/www/html;
#    }

#    location / {
#        proxy_pass http://localhost:8802;
#        proxy_set_header Host $host;
#        proxy_set_header X-Real-IP $remote_addr;
#        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
#        proxy_set_header X-Forwarded-Proto $scheme;
#    }
```

#}

Following this, Nginx's state was verified, Nginx was reloaded, and index.js was initiated with the following commands being run to finally result in a healthy API.

sudo nginx -t

sudo systemctl reload nginx

cd /home/ec2-user/apps/ClockWorkWeb/lab/backend

node index.js

Problem 2: Layout.tsx – Incorrect Import Names

Following the fix of the first problem, the logs left by Amplify during deployment of the web app uncovered another issue, unrecognized import names, as detailed below:

182 src/components/Layout/Layout.tsx

183 Line 73:44: 'Roster' is not defined react/jsx-no-undef

184 Search for the keywords to learn more about each error.

185 2025-10-06T07:50:15.953Z [ERROR]: !!! Build failed

186 2025-10-06T07:50:15.953Z [ERROR]: !!! Error: Command failed with exit code 1

This error was straightforward, with the fix simply changing:

import Roster from "../../pages/Rosters/Rosters";

...

<Route path="/rosters" element={<Roster />} />

To:

import RostersPage from "../../pages/Rosters/Rosters";

...

<Route path="/rosters" element={<RostersPage />} />

This lead to the problem not occurring anymore, however in its place another issue began revealing itself, with the following returned in the Amplify logs:

181 2025-10-06T08:12:09.079Z [INFO]: Failed to compile.

182 2025-10-06T08:12:09.085Z [INFO]: Module not found: Error: Can't resolve '../../pages/ShiftsView/ShiftsView' in '/codebuild/output/src3313065107/src/ClockWorkWebLabbing/frontend/src/components/Layout'

183 2025-10-06T08:12:09.169Z [ERROR]: !!! Build failed

184 2025-10-06T08:12:09.169Z [ERROR]: !!! Error: Command failed with exit code 1

This issue was much more troublesome for several reasons. For reasons likely attributable to multiple people working on this specific piece of code at the same time, it seemed that changes made to the names of the files or directories did not take place for a couple of hours, leading to slowed progress in the fixing of the issue. Furthermore, the problem changed at one point, with its second form seen below:

2025-10-07T03:06:18.310Z [INFO]: Module not found: Error: Can't resolve '../../pages/Shifts/Shifts' in '/codebuild/output/src1009776090/src/ClockWorkWebLabbing/frontend/src/components/Layout'

Eventually, after other changes were made to the GitHub files by other group members, I was finally able to correctly name the imports to resolve both issues as demonstrated below:

```
import ShiftsView from "../../pages/shifts/ShiftsView";

import Shifts from "../../pages/shifts/Shifts";
```

Weirdly however, although fixing this issue allowed Amplify to successfully deploy the web app, it broke certain components of it. Consequently, later on another team member somehow renamed the imports and some directories to allow both Amplify to successfully deploy the web app, and for it to not break some of the functionality through doing so.

Problem 3: Unresponsive API (504 – Bad Gateway, Timeout)

After the resolution of the previous unresponsive API issue, another one popped up later, presenting itself initially with the same “502 – Bad Gateway” message. I sought to solve this issue by changing the following line in the api-lab-clockwork.conf file:

```
 proxy_pass http://localhost:8802;
```

To:

proxy_pass http://[::1]:8802;

However, in doing this the error message turned into “504 – Bad Gateway”. I reverted the change to the configuration file in the hopes that it would bring the original message back, but the change to 504 persisted.

To resolve this, I ran the following command:

ps aux | grep node

Which allowed me to identify that 2 node processes were trying to use the same port (8802). Considering this was causing the issue I killed the processes and started a new one which solved everything. This was done using the following commands:

sudo pkill node

sudo kill -9 19728

cd /home/ec2-user/apps/ClockWorkWeb/lab/backend

node index.js

Problem 4: Missing Identity Pool

Considering the frontend requires AWS credentials to access AWS services such as S3, an identity pool is required. This was not uncovered before as this type of access was not needed previously. As a result of the lack of this necessity, the web app returned the following error in the console:

ResourceNotFoundException: IdentityPool 'ap-southeast-2:3d72b6f1-a857-4148-8f01-ae3ab2c24c9b' not found

To resolve this, an identity pool was created using AWS Cognito. This was done with logging in with Google and Amazon in mind, as they were previously implemented as federated identity providers for the BetterAuth login, along with the Amazon Cognito user pool to suit this new need for accessing AWS services. This is why they were included as the 3 identity providers for the identity pool. Considering these choices, client details from the developer accounts made for the Clockwork MVP Web App project, were needed, including the client ID of each account.

Following the creation of this identity pool, the frontend/src/aws-exports.js file was altered to suit this new change:

"aws_cognito_identity_pool_id": "ap-southeast-2:3d72b6f1-a857-4148-8f01-ae3ab2c24c9b",

To:

"aws_cognito_identity_pool_id": "ap-southeast-2:6d5ea010-9747-46ca-a38a-afa2d0aa7403",

This meant that users could authenticate themselves using BetterAuth and the web app would no longer complain about the identity pool specifically.